

Homework 05 - Arrays, Constructors, `this`, `super` And OOP

Topics Covered

- One-dimensional arrays
- Array indexing
- Array traversal using loops
- Array of objects
- Constructors
- Default constructor
- Parameterized constructor
- Constructor overloading
- `this` keyword
- `this()` constructor call
- `super` keyword
- `super()` constructor call
- Encapsulation
- Inheritance
- Method overriding
- Basic polymorphism
- Basic abstraction idea

Instructions

1. Write every program in a separate Java file.
2. First write the expected output in your notebook, then run the program.
3. Use meaningful class names, variable names, and method names.
4. Do not copy-paste the same code. Type and understand every line.
5. Use constructors wherever object data must be initialized.
6. Use `this` when constructor parameters and instance variables have the same name.
7. Use `super` only in parent-child class examples.
8. After every program, write 2-3 lines explaining what you understood.

Part A: Short Answer Questions

1. What is an array in Java?
2. Why is array index starting from `0` ?
3. What happens if we access an invalid array index?
4. What is the difference between array length and last index?
5. Why do we use loops with arrays?
6. Can an array store objects?

7. What is a constructor?
8. Why do we use constructors?
9. What is the difference between a default constructor and a parameterized constructor?
10. What is constructor overloading?
11. Can a constructor have a return type?
12. What is the use of `this` keyword?
13. What is the difference between `this` and `this()` ?
14. What is inheritance?
15. What is the use of `extends` keyword?
16. What is the use of `super` keyword?
17. What is the difference between `super` and `super()` ?
18. Why does parent constructor execute before child constructor?
19. What is method overriding?
20. What is the difference between method overloading and method overriding?
21. What is encapsulation?
22. Why should fields usually be private?
23. What is polymorphism?
24. What is abstraction?
25. Why is OOP useful in backend development?

Part B: Find The Output

Question 1

```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {10, 20, 30, 40};  
  
        System.out.println(numbers[0]);  
        System.out.println(numbers[2]);  
        System.out.println(numbers.length);  
    }  
}
```

Question 2

```
public class Main {
    public static void main(String[] args) {
        int[] marks = {70, 80, 90};

        for (int i = 0; i < marks.length; i++) {
            marks[i] = marks[i] + 5;
        }

        for (int i = 0; i < marks.length; i++) {
            System.out.println(marks[i]);
        }
    }
}
```

Question 3

```
class Student {
    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }
}

public class Main {
    public static void main(String[] args) {
        Student[] students = new Student[2];

        students[0] = new Student("Rahul", 85);
        students[1] = new Student("Priya", 92);

        System.out.println(students[0].name);
        System.out.println(students[1].marks);
    }
}
```

Question 4

```
class Product {
    String name;
    double price;

    Product(String name, double price) {
        this.name = name;
        this.price = price;
    }

    void show() {
        System.out.println(name + " " + price);
    }
}

public class Main {
    public static void main(String[] args) {
        Product product = new Product("Laptop", 55000);
        product.show();
    }
}
```

Question 5

```
class Student {
    String name;
    int age;
    String course;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    Student(String name, int age, String course) {
        this(name, age);
        this.course = course;
    }

    void show() {
        System.out.println(name);
        System.out.println(age);
        System.out.println(course);
    }
}

public class Main {
    public static void main(String[] args) {
        Student student = new Student("Amit", 21, "Java");
        student.show();
    }
}
```

Question 6

```
class Employee {
    Employee() {
        System.out.println("Employee constructor");
    }
}

class Developer extends Employee {
    Developer() {
        System.out.println("Developer constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        Developer developer = new Developer();
    }
}
```

Question 7

```
class Employee {
    String company = "Parent Company";
}

class Developer extends Employee {
    String company = "Child Company";

    void showCompany() {
        System.out.println(company);
        System.out.println(super.company);
    }
}

public class Main {
    public static void main(String[] args) {
        Developer developer = new Developer();
        developer.showCompany();
    }
}
```

Question 8

```

class Payment {
    void pay() {
        System.out.println("Generic payment");
    }
}

class UpiPayment extends Payment {
    void pay() {
        System.out.println("UPI payment");
    }
}

public class Main {
    public static void main(String[] args) {
        Payment payment = new UpiPayment();
        payment.pay();
    }
}

```

Part C: Find And Fix The Error

Question 1

```

public class Main {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30};
        System.out.println(numbers[3]);
    }
}

```

Write:

1. What is the error?
2. Why does it happen?
3. Correct the code.

Question 2

```

class Student {
    String name;
    int age;

    Student(String name, int age) {
        name = name;
        age = age;
    }
}

```

Write:

1. What is wrong in the constructor?

2. What will be stored in object fields?
3. Correct the code using `this` .

Question 3

```
class A {  
    A() {  
        System.out.println("A");  
    }  
}  
  
class B extends A {  
    B() {  
        System.out.println("B");  
        super();  
    }  
}
```

Write:

1. What is the error?
2. Why is it an error?
3. Correct the code.

Question 4

```
class Parent {  
    void show() {  
        System.out.println("Parent");  
    }  
}  
  
class Child extends Parent {  
    void show(int number) {  
        System.out.println("Child");  
    }  
}
```

Question:

Is this method overriding or method overloading?
Explain your answer.

Question 5

```
class BankAccount {
    public double balance;
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.balance = -50000;
        System.out.println(account.balance);
    }
}
```

Write:

1. What is the design problem?
2. Which OOP concept can solve it?
3. Rewrite the class using `private` field and methods.

Part D: Coding Practice

Program 1: Array Maximum

Create an integer array of 5 numbers and print the largest number.

Example:

```
Input array: 12, 45, 23, 67, 34
Output: 67
```

Program 2: Count Even And Odd Numbers

Create an integer array and count how many numbers are even and how many are odd.

Expected output format:

```
Even count: 3
Odd count: 2
```

Program 3: Student Array

Create a `Student` class with:

- `name`
- `marks`
- parameterized constructor
- `showDetails()` method

In `main`, create an array of 3 `Student` objects and print details of all students using a loop.

Program 4: Product Bill

Create a `Product` class with:

- `name`
- `price`

- `quantity`
- parameterized constructor
- `calculateTotal()` method
- `showBill()` method

Create 3 products and print the total bill amount.

Program 5: Constructor Overloading

Create an `Employee` class with:

- `id`
- `name`
- `salary`

Create two constructors:

```
Employee(int id, String name)
Employee(int id, String name, double salary)
```

If salary is not provided, store salary as `15000`.

Program 6: `this()` Practice

Create a `Course` class with:

- `courseName`
- `duration`
- `fees`

Create two constructors:

```
Course(String courseName, int duration)
Course(String courseName, int duration, double fees)
```

Call the first constructor from the second constructor using `this()`.

Program 7: Inheritance Practice

Create a parent class `Vehicle` with:

- `brand`
- `speed`
- constructor
- `showVehicleDetails()` method

Create a child class `Car` with:

- `fuelType`
- constructor
- `showCarDetails()` method

Use `super()` to call parent constructor.

Program 8: Method Overriding

Create a class `Account` with method:

```
void calculateInterest()
```

Create two child classes:

- SavingsAccount
- CurrentAccount

Override `calculateInterest()` in both child classes.

Program 9: Encapsulation

Create a `BankAccount` class with private fields:

- `accountNumber`
- `balance`

Create methods:

- `deposit(double amount)`
- `withdraw(double amount)`
- `getBalance()`

Rules:

- Deposit only positive amount.
- Withdraw only if balance is enough.
- Direct balance access should not be allowed.

Program 10: Mini OOP System

Create an online learning system.

Parent class:

```
User
- name
- email
- constructor
- showProfile()
```

Child class 1:

```
Student extends User
- courseName
- constructor
- showProfile()
```

Child class 2:

```
Trainer extends User
- subject
- constructor
- showProfile()
```

Requirements:

1. Use `super()` in child constructors.

2. Use `super.showProfile()` inside child `showProfile()`.
3. Create one `Student` object and one `Trainer` object.
4. Print both profiles.

Part E: Concept Mapping

Complete the table:

Situation	Concept Used
Protecting balance from direct access	
Child class reusing parent class fields	
Same method name with different parameters	
Same method name in parent and child class	
Calling parent constructor from child constructor	
Referring to current object field	
Storing many student objects together	
Hiding internal implementation details	

Part F: Real-World Thinking Questions

1. In a food delivery app, what classes can you identify?
2. In a banking app, why should account balance be private?
3. In an e-learning app, why can `Student` and `Trainer` extend `User`?
4. In a payment system, why can `pay()` behave differently for UPI, card, and cash?
5. In a college system, where can we use arrays of objects?

Submission Checklist

Before submitting, check:

- Did you write expected output before running output questions?
- Did you use constructors properly?
- Did you use `this` where parameter and field names are same?
- Did you use `super()` in inheritance constructor examples?
- Did you use `super.methodName()` where parent method must also run?
- Did you keep fields private in encapsulation examples?
- Did you write explanation after each program?